

Evaluación: Primera	Fecha: 14/12/2023	Calificación	
Módulo/Materia: Programación	Curso: DAW 1º		
Departamento: Informática			
Nombre y apellidos:			

Instrucciones:

- 1) Se califica desde 0 a 10 puntos.
- 2) Si aparecen ejemplos de salidas de los distintos programas, la solución tiene que ser idéntica a la del ejemplo.
- 3) En la parte práctica del examen se permiten apuntes pero no acceso a Internet ni contacto con compañeros.
- 4) Descarga la carpeta examen de la plataforma Aeducar. Dentro de la carpeta encontrarás información que puede ser útil para el examen.
- 5) Crea el proyecto ApellidoEx1ev (por ejemplo, PerezEx1ev). Al finalizar el examen, comprime en un .zip la carpeta del proyecto y la entregarás a través de la plataforma Aeducar del CPIFP Los Enlaces, en el enlace que se abrirá a tal efecto. **ES RESPONSABILIDAD TUYA ENVIAR CORRECTAMENTE LA SOLUCIÓN.**

Escribir los programas en JAVA que se indican a continuación:

Ejercicio 1: Sudoku (3.5 puntos)

Un Sudoku es un rompecabezas numérico y lógico que consiste en un tablero de 9x9 celdas dividido en subgrupos más pequeños de 3x3 llamados subcuadros o regiones. El objetivo del juego es rellenar todas las celdas del tablero con dígitos del 1 al 9, de manera que cada fila, columna y subcuadro contengan todos los números del 1 al 9 sin repetirse. En Java, un Sudoku se puede representar mediante una matriz bidimensional de números enteros donde cada número representa el valor en una celda del tablero. Se entenderá por lo tanto, que un sudoku es una matriz bidimensional de tamaño 9x9.

No hay restricciones en los métodos a utilizar para realizar este ejercicio. Se propone la creación de los siguientes métodos (funciones o procedimientos) para realizar el ejercicio:

1. **Método `mostrarSudoku`** : Tiene como parámetro un sudoku (matriz bidimensional de enteros 9x9) y lo muestra como sigue a continuación **(0.75 puntos)**

```

5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
-----
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
-----
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9

```

2. **Método `comprobarFilas`**: Tiene como parámetro un sudoku y devuelve `true` si todos los valores de las filas del sudoku son distintos y van desde el 1 al 9 y `false` en caso contrario.

(0.75 puntos)

3. **Método `comprobarColumnas`:** Tiene como parámetro un sudoku y devuelve `true` si todos los valores de las columnas del sudoku son distintos y van desde el 1 al 9 y `false` en caso contrario. **(0.75 puntos)**
4. **Método `comprobarSubcuadros`:** Tiene como parámetro un sudoku y verifica que todos los subcuadrados 3x3 del sudoku tienen valores del 1 al 9 sin repetirse. Devuelve `true` si todos los subcuadrados tienen valores del 1 al 9 sin repetirse y `false` en caso contrario. Hay un total de 9 subcuadrados (se muestran claramente en el método `mostrarSudoku`) **(0.75 puntos)**
5. **Método `main`:** Se proporciona un sudoku y nos muestra por pantalla si el sudoku es válido o no. Para la realización de este método, se puede suponer que los métodos creados funcionan correctamente (aunque no estén creados). Puedes hacer un copia-pegar del sudoku (válido) que se propone a continuación: **(0.5 puntos)**

```
int[][] sudoku = {  
    {1, 4, 7, 9, 2, 3, 8, 5, 6},  
    {6, 3, 2, 8, 7, 5, 4, 9, 1},  
    {9, 5, 8, 1, 6, 4, 3, 7, 2},  
    {8, 7, 5, 4, 1, 6, 9, 2, 3},  
    {4, 1, 9, 3, 5, 2, 6, 8, 7},  
    {2, 6, 3, 7, 9, 8, 5, 1, 4},  
    {3, 9, 4, 6, 8, 7, 1, 2, 5},  
    {5, 2, 6, 2, 4, 1, 7, 3, 8},  
    {7, 8, 1, 5, 3, 9, 2, 6, 4}  
};
```

Ejercicio 2: (3 puntos)

Crea un método llamado `generaPassword()` que devuelva un String con las siguientes características: (1,5 puntos)

- Longitud aleatoria entre 8 y 12 caracteres.
- Compuesto por caracteres en minúscula aleatorios a los que se añade en posiciones aleatorias y no repetidas.
- Una letra mayúscula.
- Dos dígitos numéricos, entre 0 - 9
- Un carácter especial, entre `{@#$%&*}`

Prueba el método anterior en un procedimiento `main` que realice las siguientes tareas (1,5 puntos)

MENU

1-Generar Password

2-Mostrar último password generado

0-Salir

Opción 1: Generará un Password y volverá a mostrar el menú.

Opción 2: Mostrará el último Password generado y volverá a mostrar el menú.

```
MENU
1-Generar Password
2-Mostrar último password generado
0-Salir
2
Password: yxp#0hVdy8n
MENU
1-Generar Password
2-Mostrar último password generado
0-Salir
```

Opción 0: Saldrá del programa e indicará el número total de claves generadas y mostrará todas las claves que ha ido generando.

```
MENU
1-Generar Password
2-Mostrar último password generado
0-Salir
0
Claves Generadas: 7
password 1: yxp#0hVdy8n
password 2: uv10t#zñ6uF
password 3: 3z@ggfdzWk4
password 4: ocauI4#7ni
password 5: r7By&tf5l
password 6: pñq44zl@Hujx
password 7: g%Benfbr4qi7
Fin del programa
```

Otra opción: Mostrará “Opción Incorrecta” y volverá a mostrar el menú.

```
MENU
1-Generar Password
2-Mostrar último password generado
0-Salir
8
Opción Incorrecta
MENU
1-Generar Password
2-Mostrar último password generado
0-Salir
```

Ejercicio 3: (2 puntos)

Genera un programa que te pida números hasta que introduzcas el valor 0. El valor 0 no debe ser incluido. (0,25 puntos)

Almacena los números introducidos en dos arrays de enteros `numerosPrimos` y `numerosNoPrimos`, según el número introducido sea primo o no. (0,75 puntos)

Muestra la cantidad de números que se han introducido repetidos y los vectores ordenados de mayor a menor. (1 punto).

Nota:

Si se introduce el mismo número dos veces o más no tiene que aparecer en el vector.

Salida:

```
Número: 2
Número: 2
Número: 4
Número: 5
Número: 6
Número: 6
Número: 9
Número: 9
Número: 7
Número: 10
Número: 11
Número: 13
Número: 0
INFORME
Total de números repetidos: 3
Primos: (13<>11<>7<>5<>2)
No Primos: (10<>9<>6<>4)
```

Ejercicio 4: (2 puntos)

Genera un programa que simule el juego del 7.

En el juego dispones de una cantidad inicial de 50 Euros para apostar.

El juego termina cuando te quedas sin dinero o decides abandonar el juego.

Aparecerá el siguiente menú:

```
MENU
1-Tirar Dados
2-Mostrar cantidad
0-Salir
```

Opción 1: Se pide la cantidad a apostar (no puede ser inferior a 0 ni superior a la cantidad que se tenga en ese momento y se lanzan los dados (se deben de mostrar los dados)

Si al lanzar los dados la suma obtenida es 7, ganas y obtienes una cantidad igual a la apostada. En caso contrario, pierdes lo apostado.

Crea un método tirarDados() que devuelva un entero que sea la suma de los dos datos y que simule y muestre tirar dos dados.

```
MENU
1-Tirar Dados
2-Mostrar cantidad
0-Salir
1
Cantidad a apostar:
20
6***1
MENU
1-Tirar Dados
2-Mostrar cantidad
0-Salir
```

Si te quedas sin dinero termina el juego, mostrando un mensaje.

```
MENU
1-Tirar Dados
2-Mostrar cantidad
0-Salir
1
Cantidad a apostar:
30
3***6
No te queda cantidad para apostar
Fin del Juego
```

Opción 2: Se motrará la cantidad disponible.

```
MENU
1-Tirar Dados
2-Mostrar cantidad
0-Salir
2
La cantidad disponible es: 30
MENU
1-Tirar Dados
2-Mostrar cantidad
0-Salir
```

Opción 0: Se motrará la cantidad disponible y se saldrá del juego.

```
MENU
1-Tirar Dados
2-Mostrar cantidad
0-Salir
0
La cantidad total es: 30
Fin del Juego
```